

Smartband V001 — Rev1 Firmware Development

A Detailed Development Log: From Board-File Mismatches to a Working BLE-Enabled Sensor Firmware

Masrur Jamil Prochchhod · Matriculation 2210028 Hochschule Hamm-Lippstadt (HSHL) · Supervisor: Prof. Giovanni

1. Introduction and Project Context

Smartband V001 is a custom Bluetooth Low Energy (BLE) health wearable being developed at HSHL. Its current hardware revision, Rev1, is a custom PCB built around the Texas Instruments CC2640R2F BLE System-on-Chip, and carries three sensors connected over a shared I2C bus:

- LIS331DLTR — a 3-axis MEMS accelerometer (I2C address 0x18)
- MAX30205 — a clinical-grade digital temperature sensor (I2C address 0x48)
- MAX30102 — an integrated pulse-oximetry / heart-rate sensor providing raw IR and red LED photodiode readings (I2C address 0x57)

A Raspberry Pi 5 acts as a companion processing device, but the work described in this document is confined entirely to the embedded firmware running on the CC2640R2F itself — the Raspberry Pi / dashboard software is covered separately and intentionally omitted here.

This document exists because the project entered this development phase in a difficult position: Rev1's Bluetooth stack had been failing for some time with a hardware exception fault inside `GAPRole_StartDevice()`, a core Bluetooth stack initialization function. A previous investigation into this fault had consumed roughly twenty hours without resolution, and the working theory at the time was that this represented a genuine RF or hardware defect on the custom board — potentially meaning the radio circuitry itself was faulty. Given how serious a conclusion that would be for a custom PCB design, it was essential to investigate this properly, from first principles, rather than continuing to guess at fixes on top of an already-tangled, roughly 1,700-line legacy firmware file that mixed BLE stack code, sensor code, and OLED display code together.

The approach taken, and documented in full below, was to abandon debugging the old tangled file entirely and instead rebuild the firmware from the cleanest possible starting point, verifying one single capability at a time on real hardware before adding the next. This is what ultimately uncovered the true root cause — and it turned out to have nothing to do with the radio hardware at all.

2. Development Environment and Tools

Tool / Component	Role in this Work
Code Composer Studio (CCS) 12.8.1	IDE used for all firmware editing, building, flashing, and hardware debugging
TI SimpleLink CC2640R2 SDK v5.30.1.11	Source of the board files, drivers, and BLE5-Stack example projects used as clean starting points
TI LAUNCHXL-CC2640R2 LaunchPad (XDS110 debug probe)	Used purely as an external JTAG programmer/debugger connected to Rev1's P1 header — the LaunchPad's own onboard MCU is never running any of this project's code
CCS Hardware Debugger	Breakpoints, Suspend/Resume, call-stack inspection, and disassembly view — the primary diagnostic tool throughout this work
Raspberry Pi 5 (UART, /dev/ttyAMA0, 115200 baud)	Used only as a serial terminal to observe live firmware output during development — not part of the firmware itself
nRF Connect (mobile BLE scanner/client)	Used to independently verify BLE advertising and connection behaviour from outside the firmware

Table 1 — Tools and their role in this development process.

3. Methodology — Why a Clean Rebuild, Not a Patch

Given that the prior fault investigation had already consumed significant time without success, and that the existing firmware conflated at least three unrelated subsystems (BLE stack, three different sensor drivers, and an OLED display driver) inside a single very large file, a decision was made to change strategy entirely rather than continue patching the same file:

- Start from TI's own, completely unmodified SDK example projects — first a bare TI-RTOS project with no Bluetooth stack at all, and later TI's official ble5stack "simple_peripheral" example — rather than the project's existing legacy code.
- Add exactly one new capability at a time: first confirm the RTOS itself boots, then UART, then the I2C bus, then each individual sensor, and only combine everything into a periodic loop once every individual piece had been proven correct in isolation.
- After every single change, rebuild, reflash the physical board, and observe real behaviour — either via UART text output on the Raspberry Pi terminal, or via the CCS hardware debugger when something did not work as expected.
- Treat every unexpected hang or fault as a debugging opportunity rather than something to work around: use CCS's Suspend function mid-execution, inspect the call stack to see exactly which function the processor was inside, and use the Disassembly view when source-level information was not available (for example, inside ROM-resident driver library code).
- Deliberately delay reintroducing the Bluetooth stack until the sensor firmware was fully working, specifically so that if the original BLE fault reappeared, it would be possible to know for certain whether it was a BLE-specific problem or something more fundamental that BLE merely happened to expose.

This incremental, hardware-verified approach is what allowed each individual problem to be isolated cleanly, rather than several overlapping problems being investigated at once — which is very likely what made the original fault so difficult to resolve in the first twenty-hour investigation.

4. Building the Sensor Firmware, Stage by Stage

4.1 Stage 1 — Confirming the RTOS Boots Cleanly

The very first step was to prove that the CC2640R2F chip on Rev1, and the JTAG connection to it through the LaunchPad's XDS110 debug probe, were both fundamentally healthy — completely independent of any Bluetooth code. For this, TI's built-in "empty" TI-RTOS driver example was imported directly from the SDK for the CC2640R2_LAUNCHXL target, and built and flashed onto Rev1 completely unmodified.

This project does essentially nothing: it calls Board_init(), creates a single background thread, and that thread sits in an infinite loop. That simplicity was the entire point — if this alone failed to run, the problem would have to be something fundamental about the chip, the JTAG connection, or the power/ground wiring, rather than anything BLE-related.

Result: the project built with zero errors and, once flashed, ran without any fault. Using the CCS debugger's Suspend button at several different points in time, the processor was consistently found sitting inside the RTOS idle task — specifically inside a function called CPUwfi() ("CPU Wait For Interrupt"), invoked via PRCMDeepSleep(). This is the normal, expected behaviour of an idle TI-RTOS system with nothing else to do, and its presence — rather than a hard fault or an unexpected address — was the first solid, positive confirmation that the chip and debug connection were working correctly.

4.2 Stage 2 — Adding UART Output, and the First Real Bug

With the RTOS baseline established, the next step was to get any kind of visible output from the board, to make every subsequent stage easier to verify. A simple UART transmit routine was added to print an incrementing counter value once every three seconds, to be read on a Raspberry Pi via a serial terminal at 115200 baud.

On the very first attempt to flash and run this, nothing was ever received on the Raspberry Pi — not even once. Using the CCS debugger's Suspend function, the call stack revealed that the processor had never actually reached the application code at all. It was stuck indefinitely inside a function called `PIN_init()`, specifically inside a `while(1)` loop that only executes if pin initialization failed:

```
void CC2640R2_LAUNCHXL_initGeneral(void)
{
    Power_init();

    if (PIN_init(BoardGpioInitTable) != PIN_SUCCESS) {
        /* Error with PIN_init */
        while (1);
    }

    Board_initHook();
}
```

This meant `PIN_init()` was returning failure — something in the pin configuration table it was given could not be successfully claimed by the chip. The array in question, `BoardGpioInitTable`, is defined inside the SDK's stock board file (`CC2640R2_LAUNCHXL.c`), and lists every GPIO pin the board needs configured at startup. Inspecting its contents line by line revealed the actual problem:

```
const PIN_Config BoardGpioInitTable[] = {
    CC2640R2_LAUNCHXL_PIN_RLED | PIN_GPIO_OUTPUT_EN | ..., /* Red LED */
    CC2640R2_LAUNCHXL_PIN_GLED | PIN_GPIO_OUTPUT_EN | ..., /* Green LED */
    CC2640R2_LAUNCHXL_PIN_BTN1 | PIN_INPUT_EN | ..., /* Button 1 */
    CC2640R2_LAUNCHXL_PIN_BTN2 | PIN_INPUT_EN | ..., /* Button 2 */
    CC2640R2_LAUNCHXL_SPI_FLASH_CS | PIN_GPIO_OUTPUT_EN | ..., /* Ext. SPI flash CS */
    CC2640R2_LAUNCHXL_UART_RX | PIN_INPUT_EN | ...,
    CC2640R2_LAUNCHXL_UART_TX | PIN_GPIO_OUTPUT_EN | ...,
    CC2640R2_LAUNCHXL_SPI0_MOSI | PIN_INPUT_EN | ...,
    CC2640R2_LAUNCHXL_SPI0_MISO | PIN_INPUT_EN | ...,
    CC2640R2_LAUNCHXL_SPI0_CLK | PIN_INPUT_EN | ...,
    PIN_TERMINATE
};
```

Every one of the entries above — the red and green LEDs, both push-buttons, the external SPI flash chip select, and the SPI bus pins used to talk to that flash chip — describes hardware that exists on TI's reference LaunchPad development board, but does not exist anywhere on Rev1's custom PCB. Rev1 has no onboard LEDs, no buttons, and no external SPI flash chip. When `PIN_init()` tried to configure pins for hardware that was never populated, on at least one of those pins the configuration could not be validated, and the whole call failed — which meant the firmware never progressed past this single line, no matter what the rest of the application code did.

The fix was to strip this table down to only the pins Rev1's actual PCB uses:

```
const PIN_Config BoardGpioInitTable[] = {
    /* Rev1 has no onboard LEDs, buttons, or SPI flash -- only UART
     * RX/TX kept (needed for debug backchannel / our own use). */
    CC2640R2_LAUNCHXL_UART_RX | PIN_INPUT_EN | PIN_PULLDOWN,
    CC2640R2_LAUNCHXL_UART_TX | PIN_GPIO_OUTPUT_EN | PIN_GPIO_LOW | PIN_PUSHPULL,
    PIN_TERMINATE
};
```

After rebuilding and reflashing with this change, execution progressed past `PIN_init()` — but a second, different hang immediately appeared. This time, Suspend showed the processor stuck inside `PIN_close()`, reached through a chain of calls: `Board_initHook()` → `CC2640R2_LAUNCHXL_shutDownExtFlash()` → `CC2640R2_LAUNCHXL_wakeUpExtFlash()` → `PIN_close()`. Looking at the actual source of these functions (found via CCS's "Open Declaration" feature, since a text search initially failed to locate them — they live in a separate file, `CC2640R2_LAUNCHXL_fxns.c`) made the cause clear:

```
void Board_initHook()
{
```

```
CC2640R2_LAUNCHXL_shutDownExtFlash();  
}
```

This function's entire job is to wake up an external SPI flash chip via a specific chip-select toggle sequence, then immediately put it back to sleep — again, this is routine housekeeping for the LaunchPad's onboard flash chip. Since Rev1 has no such chip on those pins at all, the wake/shutdown handshake this code expects never receives the response it is waiting for, and `PIN_close()` ends up spinning forever trying to release pin state that never resolves the way the driver expects.

The fix here was simply to make `Board_initHook()` do nothing at all, since Rev1 requires no board-specific post-initialization steps:

```
void Board_initHook()  
{  
    /* Intentionally empty -- no ext flash on Rev1 */  
}
```

With both of these fixes in place, the firmware was rebuilt and reflashed once more. This time, the counter message was received cleanly on the Raspberry Pi terminal every three seconds, exactly as intended — the first real, visible confirmation that application-level code was executing correctly on Rev1.

4.3 Stage 3 — I2C Bus and the LIS331DLTR Accelerometer

The next stage added I2C communication and attempted the simplest possible sensor test: reading the LIS331DLTR accelerometer's `WHO_AM_I` identification register, which should return a fixed, known value if the chip is being addressed correctly.

Before this could work, the same category of board-file mismatch from Stage 2 had to be checked for again — and it was present. The SDK's default I2C pin defines, `CC2640R2_LAUNCHXL_I2C0_SCL0` and `CC2640R2_LAUNCHXL_I2C0_SDA0`, resolve to `IOID_4` and `IOID_5` respectively — the pin numbers used for I2C on the stock LaunchPad. Rev1's actual schematic places SCL on `IOID_5` and SDA on `IOID_6`, a different pair entirely. This mismatch had to be corrected in two separate places: the GPIO initialization table, and the I2C peripheral driver's own hardware-attributes structure, since both independently reference these pin numbers:

```
/* In BoardGpioInitTable: */  
/* Rev1 I2C bus -- SCL=DIO_5 (P1 pin 6), SDA=DIO_6 (P1 pin 8).  
 * NOT using CC2640R2_LAUNCHXL_I2C0_SCL0/SDA0 -- wrong for Rev1. */  
IOID_5 | PIN_INPUT_EN | PIN_PULLUP, /* SCL */  
IOID_6 | PIN_INPUT_EN | PIN_PULLUP, /* SDA */  
  
/* In the I2CCC26XX_HWAttrsV1 struct: */  
.sdaPin = IOID_6,  
.sclPin = IOID_5,
```

With both corrections applied, a single `WHO_AM_I` register read was performed and printed over UART. The value returned was `0x32`, consistently, across many repeated test runs. This value was independently cross-checked and confirmed to be the correct, documented identification value for the LIS331DLH/LIS331DLTR accelerometer family — strong evidence that the physical I2C bus, the pin wiring, and the register-level read logic were all functioning correctly together, not just that some byte happened to come back.

4.4 Stage 4 — MAX30205 Temperature Sensor

With the I2C bus proven functional, a second sensor was brought online: the MAX30205 digital temperature sensor at I2C address `0x48`. Its temperature register was read as a raw 16-bit two's-complement value and converted to degrees Celsius using the sensor's documented scale factor of $1/256$ °C per count:

```
float temp_c = temp_raw / 256.0f;
```

The resulting readings were stable at approximately 26.8–26.9 °C across repeated samples taken a few seconds apart — a physically reasonable room-temperature value, with only the small amount of variation expected from real

sensor noise and minor self-heating, not the wildly inconsistent or frozen values that would indicate a bus or addressing problem.

4.5 Stage 5 — MAX30102 Pulse-Oximetry Sensor

The third and final sensor, the MAX30102, was verified the same way: by reading its Part ID register (register address 0xFF), which the MAX3010x product family uses specifically so that software can distinguish between otherwise similar sibling parts (the MAX30100, MAX30101, and MAX30105 all share the same family but have different Part ID values). The read consistently returned 0x15 — the documented, correct Part ID specifically for the MAX30102, confirming the correct physical part was present and responding correctly on the bus.

4.6 Stage 6 — From One-Shot Diagnostics to Continuous, Combined Operation

With all three sensors individually verified via simple identification-register reads, the firmware was converted into something closer to its intended final form: a continuously running loop reading real, changing sensor data from all three devices together, printed in clearly labelled form once every three seconds, using TI-RTOS's built-in Clock/Util_startClock periodic-timer mechanism rather than a manual busy-wait loop.

Two refinements were made at this stage that meaningfully improved the quality and trustworthiness of the data being produced.

A. Fixing torn accelerometer reads

The accelerometer's X-axis value is stored across two separate registers — a low byte (OUT_X_L) and a high byte (OUT_X_H) — which must be combined to form one 16-bit reading. The first version of this code read each register with its own, separate I2C transaction:

```
/* BEFORE -- two separate transactions, can "tear" across samples */
int16_t get_i2c_value_lis331_accel_x(void)
{
    uint8_t xl = get_i2c_value_lis331_xl(); /* transaction #1 */
    uint8_t xh = get_i2c_value_lis331_xh(); /* transaction #2 */
    int16_t raw16 = (int16_t)((xh << 8) | xl);
    return (int16_t)(raw16 >> 4);
}
```

Because the accelerometer's internal registers keep updating continuously between these two separate I2C transactions, it is possible for the low byte from one physical sample to be combined with the high byte from a completely different, later sample — a classic "torn read." This was in fact observed directly: the board sitting perfectly still on a desk still produced large, physically meaningless swings in the reported acceleration value. The fix was to read both bytes in a single I2C transaction, using the accelerometer's auto-increment addressing feature (setting the high bit of the register address causes the chip to automatically advance to the next register for each additional byte requested):

```
/* AFTER -- single burst read, both bytes from the same sample */
int16_t get_i2c_value_lis331_accel_x(void)
{
    ...
    tx_buf[0] = LIS331_OUT_X_L | 0x80; /* auto-increment bit set */
    i2c_trans.writeCount = 1;
    i2c_trans.readCount = 2; /* reads OUT_X_L then OUT_X_H atomically */
    ...
    int16_t raw16 = (int16_t)((rx_buf[1] << 8) | rx_buf[0]);
    return (int16_t)(raw16 >> 4);
}
```

After this change, the accelerometer reading settled into a small, stable range of roughly 10–18 milli-g while the board sat stationary — consistent with ordinary MEMS sensor noise rather than measurement corruption — and changed meaningfully and predictably whenever the board was deliberately tilted or moved.

B. Replacing static identity values with live, changing data

Once the bus itself was proven reliable, the diagnostic WHO_AM_I and Part ID checks (which never change, by design) were replaced with genuinely live sensor readings for demonstration purposes: the accelerometer's X-axis in milli-g, and the MAX30102's raw infrared photodiode reading taken directly from its FIFO_DATA register. The latter was confirmed to behave exactly as expected of a real optical sensor: under normal room lighting it read a moderate baseline value (dominated by ambient infrared light reaching the photodiode), and when the sensor package was deliberately covered by a fingertip, the reading changed dramatically, confirming the LED and photodiode were both physically functioning and that the I2C link was delivering genuinely live, physically meaningful data rather than a frozen or default value.

5. Investigating and Resolving the Original BLE Fault

With a fully working, individually-verified sensor firmware now in hand, the original, long-standing question could finally be investigated properly: why did the Bluetooth stack fault with a hardware exception inside GAPRole_StartDevice()?

5.1 Isolating BLE From Everything Else

Rather than attempt to debug this inside the old, tangled application file, a second clean project was built: TI's own, completely unmodified ble5stack "simple_peripheral" example, imported directly from the SDK. This example project consists of two separate CCS projects that must both be built — an application project and a separate BLE stack library project — which was confirmed and built in the correct order (library first, then application) before any testing began.

On the very first attempt to run this project, exactly the same fault signature already diagnosed and fixed in Stage 2 reappeared: execution hung inside PIN_init(), then, after the same fix was applied, inside PIN_close() via Board_initHook(). This was an extremely important finding in its own right — it proved conclusively that these were not bugs specific to the sensor firmware, but generic defects present in any project built against the SDK's stock CC2640R2_LAUNCHXL board files when used with Rev1's custom hardware.

The identical set of fixes already validated in Stages 2 and 3 — the trimmed GPIO table, the emptied Board_initHook(), and the corrected I2C pin assignment — was applied to this BLE project's own copy of the board files. After rebuilding both the stack library and application projects and reflashing, the firmware advanced cleanly past every point that had previously blocked execution, and the CCS debugger showed it settling into normal RTOS idle behaviour (the same CPUwfi/PRCMDDeepSleep pattern seen back in Stage 1) rather than faulting.

5.2 Independent Confirmation via nRF Connect

Code-level confirmation that the firmware was not crashing was reinforced with an entirely independent, external test: the nRF Connect mobile application was used to scan for nearby Bluetooth Low Energy devices. The device advertised successfully under its configured name, "Simple Peripheral," and a full BLE connection was established and held from the phone — something that would be architecturally impossible if the underlying radio hardware were genuinely defective, since a faulty RF front-end would prevent advertising packets from ever being transmitted or received correctly in the first place.

5.3 Conclusion — the Original Diagnosis was Wrong

This directly and conclusively overturns the previous working theory. The CC2640R2F radio hardware on Rev1 is fully functional. The original fault reported inside GAPRole_StartDevice() was never an RF or silicon defect — it was the same class of board-configuration-file mismatch identified and fixed in Stage 2, simply occurring earlier in the overall boot sequence than the point where the fault was eventually observed and reported. In practice, this means the prior twenty-hour investigation had likely never actually reached the real Bluetooth stack code at all — it had been blocked, the entire time, on generic board initialization code common to any project built from these SDK board files.

6. Combining the Verified Sensor Code with the Working BLE Project

With both halves independently proven — sensors working in one clean project, BLE working in another — the final step was to merge them: the already-correct sensor driver files (`i2c_sensors.c` and `i2c_sensors.h`) were copied into the working BLE project, and the periodic sensor-reading logic from Stage 6 was added inside the BLE application's own existing periodic-task function, `SimplePeripheral_performPeriodicTask()`, which the BLE stack already calls automatically on a timer.

This integration step, despite reusing already-proven code on both sides, surfaced several new and instructive problems of its own, each resolved through the same systematic, one-variable-at-a-time debugging approach used throughout this project.

6.1 An Include-Order Header Collision

After adding `#include "i2c_sensors.h"` to the BLE project's main application file, the build began failing with an error claiming that `SBP_DISPLAY_TYPE` — a macro used to select which display driver back-end to use, and one that had compiled correctly moments earlier — was suddenly "undefined," even though nothing related to that macro had been touched.

Investigation revealed the cause: `i2c_sensors.h` itself contains `#include "Board.h"` (with a capital B). The BLE project separately includes its own board header via `#include <board.h>` (lowercase). On a case-insensitive filesystem such as Windows, these two include statements refer to the exact same physical file. Because C header files use "include guards" to prevent being processed twice, whichever of these two inclusions happened first would cause the second, differently-capitalized inclusion to be silently skipped by the preprocessor — and depending on the exact order in which this happened, code that `SBP_DISPLAY_TYPE`'s definition depended on could end up not being processed in time. The fix was simply to remove the unnecessary, redundant `#include "Board.h"` line from inside `i2c_sensors.h` — it was already included separately, correctly, inside `i2c_sensors.c` where it is actually needed, and did not need to also live in the header file at all.

6.2 A Missing Build-Time Macro

Separately, it emerged that `SBP_DISPLAY_TYPE` is not actually defined anywhere in this project's own source code at all — it is normally supplied to the compiler as a command-line predefined symbol, configured as part of the project's build settings, pointing at whichever physical display back-end (in this case, UART-based text output) the board is configured to use. For reasons not fully identified — quite possibly disturbed by the extensive Clean/rebuild cycles performed while diagnosing the header collision above — this predefined symbol stopped reaching the compiler for this particular build configuration.

Rather than continue chasing an elusive project-settings misconfiguration, a small, explicit, guarded definition was added directly to the source file, which resolves the issue permanently regardless of whatever the underlying build-settings problem was:

```
#ifndef SBP_DISPLAY_TYPE
#define SBP_DISPLAY_TYPE Display_Type_UART
#endif
```

6.3 A Malformed Comment That Silently Deleted Real Code

During one round of edits, a closing block-comment marker was accidentally typed as `/* --- END TEMP I2C TEST --- *` (missing the final forward slash), rather than the correct `*/`. In the C language, a comment that is never properly closed does not stop at the end of the line — it continues consuming everything that follows, including real, working code, until the next `*/` sequence appears anywhere later in the file. In this case, that swallowed several lines of genuine application code, including the initialization of the Two-Button Menu module, producing confusing compiler errors reported many lines further down the file than the actual mistake. This was traced by carefully re-examining the exact file contents around the reported error lines rather than assuming the error message's line

number pointed directly at the real problem — a useful general lesson, since a single missing character in a comment can produce compiler errors that appear completely unrelated and far removed from their true cause.

6.4 A Competing Second UART Handle

As an intermediate diagnostic step, a temporary test was added that opened its own, separate raw UART handle directly, independent of the display system, to send a simple one-off test message. This compiled and ran without crashing, but its message never actually appeared over the physical UART connection — yet, confusingly, the BLE stack's own status messages (such as "Initialized" and "Adv Set 1 Enabled"), sent moments later through the project's existing `Display_printf()` mechanism, arrived perfectly correctly on the same physical UART link.

This was resolved by testing the two systems independently, back-to-back, and observing that the BLE-project's own already-working `Display_open()/Display_print1()` call path could successfully deliver output, while a second, separately-opened raw UART handle to the same physical peripheral could not. The conclusion was that the display driver had already taken ownership of the single physical UART peripheral, and a second, independent open of that same hardware resource was not a reliable way to share it. The fix was to abandon the separate raw UART handle entirely, and instead route the sensor readings through the exact same, already-proven `Display_print1(displayHandle, ...)` call used successfully elsewhere in the same file:

```
dispHandle = Display_open(SBP_DISPLAY_TYPE, NULL);

if (dispHandle != NULL) {
    I2C_init();
    uint8_t lis_whoami = get_i2c_value_lis331_whoami();
    Display_print1(dispHandle, 12, 0,
                  "TEST: LIS331_WHOAMI=0x%02x", lis_whoami);
}
```

With this change, the test message "TEST: LIS331_WHOAMI=0x32" was received correctly — the first successful sensor output from inside the BLE-enabled project.

6.5 Adding a MAX30102 Initialization Step, and the Final Periodic Loop

One further, easily overlooked step was required before the MAX30102's readings would be physically meaningful: the sensor's LED drive current and operating mode must be explicitly configured via a one-time initialization call before its FIFO data becomes useful. Without this call, the sensor's LED never actually turns on, and its readings stay at a stale or zero value regardless of how often the FIFO register is read:

```
/* One-time MAX30102 init: sets LED current + mode. Without this,
 * the LED never turns on and IR_RAW stays at 0 or a stale value
 * regardless of how often you read it. */
get_i2c_value_max3010x_basicinit();
```

With this call correctly placed once during startup, all three sensors were finally combined into a single periodic function, reading and printing live values from all three devices every three seconds, using clearly labelled output:

```
static void SimplePeripheral_performPeriodicTask(void)
{
    ...
    if (dispHandle != NULL) {
        int16_t accel_x = get_i2c_value_lis331_accel_x();
        uint16_t temp_raw = get_i2c_value_max30205_rawtemp(0x48);
        uint32_t ir_raw = get_i2c_value_max3010x_ir_raw();

        float temp_c = temp_raw / 256.0f;
        ...
        Display_print1(dispHandle, 13, 0,
                      "LIS331DLTR ACCEL X=%dmg", (int)accel_x);
        Display_print2(dispHandle, 14, 0,
                      "MAX30205 TEMP=%d.%02dC", temp_int, temp_frac);
        Display_print1(dispHandle, 15, 0,
```



```

        "MAX30102 IR_RAW=%lu", (unsigned long)ir_raw);
    }
}

```

Crucially, this periodic task is started once at boot, unconditionally, rather than only after a BLE connection is established as in the original example code — meaning the sensor readings continue updating and printing regardless of whether a phone or other BLE central device is currently connected, while the device simultaneously continues to advertise and accept BLE connections in the background.

7. Summary of All Root Causes and Fixes

Symptom Observed	Root Cause	Fix Applied
PIN_init() never returns; RTOS never reaches application code	BoardGpioInitTable configures LEDs, buttons, and SPI-flash pins that only exist on the stock LaunchPad reference board, not on Rev1	Table stripped to only Rev1's real UART and I2C pins
PIN_close() hangs forever inside Board_initHook()	Board_initHook() tries to wake/sleep an external SPI flash chip Rev1 does not have; the handshake never completes	Board_initHook() body emptied entirely
I2C reads return incorrect or no data	SDK's default I2C pin defines (IOID_4/5) match the stock LaunchPad, not Rev1's real schematic (IOID_5/6)	Corrected in both BoardGpioInitTable and the I2CCC26XX_HWAttrsV1 driver structure
Accelerometer jitters wildly even while stationary	Two separate single-byte I2C reads can sample across two different underlying sensor updates (a "torn read")	Single burst read using the accelerometer's auto-increment address bit
Original: "GAPRole_StartDevice() hardware fault" (months-old, unresolved)	Identical board-file mismatches as above, occurring earlier in the boot sequence than the reported fault location — not an RF or silicon defect	Same board-file fixes applied to the BLE project; confirmed working via nRF Connect
SBP_DISPLAY_TYPE suddenly "undefined" after adding a new #include	Case-insensitive filesystem collision between i2c_sensors.h's own Board.h include and the project's own board.h include	Removed the redundant Board.h include from i2c_sensors.h
SBP_DISPLAY_TYPE genuinely missing from the build	A compiler predefine that normally supplies this macro stopped reaching this build configuration	Added a small guarded #define directly in source as a reliable fallback
Confusing, seemingly unrelated compiler errors dozens of lines away	A block comment was closed with only "*" instead of "**/", silently absorbing real code that followed	Corrected the comment terminator; re-verified the absorbed code was intact
Raw UART test message never arrives, though BLE's own status messages do	A second, independently-opened UART handle competed with the display driver, which already owned the same physical UART peripheral	Sensor output routed through the existing, already-working Display_print1()/dispHandle path instead
MAX30102 IR reading stays at zero regardless of lighting or fingertip contact	The sensor's LED current/mode registers were never configured, so its LED never physically turns on	Added a one-time get_i2c_value_max3010x_basicinit() call during startup

Table 2 — Complete list of symptoms, root causes, and fixes identified across this development process.

8. Conclusion

This development phase replaced an unresolved, twenty-hour-old hardware mystery with a fully understood, documented, and fixed root cause, and produced two clean, working firmware artefacts along the way: a standalone sensor-verification firmware proving all three of Rev1's sensors work correctly, and a combined sensor-plus-BLE firmware proving that Rev1's Bluetooth radio hardware was never faulty in the first place.

The single unifying theme across almost every problem encountered — from the very first `PIN_init()` hang in Stage 2, through the original BLE fault itself, all the way to the header-collision issue during final integration — was a mismatch between what TI's stock SDK board files assume about the hardware (the reference LaunchPad development board) and what Rev1's actual custom PCB physically contains. Recognising this pattern early, and checking for it systematically at every new stage, is likely to remain directly useful for Rev2 and any future hardware revision of this project, since any new custom board will need the same category of board-file review before its BLE stack or sensor drivers can be trusted.

More broadly, this work demonstrates the practical value of methodical, incremental, hardware-verified debugging over attempting to fix a fault directly inside a large, multi-purpose codebase: every one of the issues documented above was found and resolved within a single working session once the decision was made to isolate each subsystem completely and test it independently on real hardware, in contrast to the twenty hours previously spent investigating the same underlying class of problem without success.